Sprint 3 — Relatório de análise crítica dos testes gerados por LLM

Projeto: P07 — Controle de Frequência · **Rodada de melhoria posterior à v1:** 09/11/2026
Branch proposta: `sprint3/evolucao-v1-v2-pos-v1`

1. Contexto e método

O grupo selecionou quatro histórias: **US01**, abrir/administrar chamada por código temporário; **US02**, registrar presença mediante código; **US03**, consultar chamadas disponíveis; e **US04**, classificar e consultar a frequência de alunos. Os **21 critérios de aceitação** foram reconstruídos a partir do enunciado e código do P07 e desdobrados em uma matriz com **48 casos (T01–T48)**, 25 partições, 16 limites e 7 fluxos. A matriz foi commitada em `e2866be7`, antes da geração dos testes. Esses critérios foram acordados pelo grupo, mas não são um documento de user stories anterior à Sprint 3.

A **v1 oficial**, gerada em 09/11 em conversa nova com Claude Opus 5.5, foi preservada em `frequencia-back/tests/v1-llm/chamada-v1.test.ts`. Seu prompt integral, que continha critérios e código mas não a matriz T01–T48, está em `docs/testes/prompt-v1.md`; a resposta possui **117 testes**. Não há garantia absoluta de isolamento de memória externa à conversa. A v1 foi gerada sobre o código já refatorado, embora também tenha passado no código original, conforme logs versionados.

A **v2 inicial**, produzida em 08/11 pelo ChatGPT a partir dos 48 casos da matriz, **precede cronologicamente a nova v1**. Essa limitação foi identificada na revisão de 09/11. Para produzir uma rodada *efetivamente posterior* à v1, foi criada uma branch independente que **manteve os 117 testes da v1 intocados**, preservou os 48 testes da v2 e acrescentou **11 testes de `FrequenciaService`**, após identificar que a v1 não exercitava esse serviço.

2. O que a v1 acertou e onde ficou a lacuna

No escopo antigo de quatro arquivos, a v1 tinha 117/117 testes aprovados e cobria todos os 48 casos da matriz: 100% de linhas e 97,91% de branches. A v2 inicial tinha 48/48, com as mesmas métricas. Assim, a principal diferença **qualitativa** era a precisão dos oráculos: **15 testes da v1** usavam `rejects.toThrow()` genérico (qualquer exceção satisfaz a asserção); a v2 verifica os erros HTTP relevantes (por exemplo, 400, 409 e 429) e possuía **0** desses oráculos genéricos. A v1 continha ainda um teste probabilístico de unicidade de códigos e verificações acopladas à forma das consultas Prisma.

A lacuna remanescente da v1 é **funcional e de escopo**: ela testa as fórmulas (`frequencia.util.ts`), mas não testa `FrequenciaService`, que consulta o aluno por turma ou e-mail, conta aulas realizadas, ordena histórico e compõe o resumo. Essa camada é relevante à US04. Não foi identificada como defeito real do produto; trata-se de **ausência de teste**.

3. Rodada nova: evolução da v2 posterior à v1

A v2 ampliada inclui os casos **EVO-01 a EVO-11**, descritos em `docs/testes/ADENDO-EVOLUCAO-V1-V2.md`: sem e-mail, e-mail vazio, nenhuma matrícula, consulta de uma ou várias turmas, ordenação temporal, aluno sem marcação, zero aulas, restrição por turma, recurso inexistente e fronteira de 25% de faltas. Os testes usam mocks do Prisma e oráculos explícitos para os resultados observáveis e consultas essenciais.

A **comparação foi feita com escopo ampliado idêntico** nas duas configurações do Vitest: `chamada-codigo.service.ts`, seus dois DTOs, `frequencia.util.ts` e `frequencia.service.ts`. O novo arquivo não foi incluído somente na v2, o que tornaria a comparação inválida. O código de produção não foi modificado nesta proposta.

4. Indicadores medidos

Indicador	v1 oficial (5 arquivos)	v2 ampliada (5 arquivos)	Ganho
Testes declarados	117	59	v2 acrescentou 11 aos seus 48 testes anteriores
Statements	84,48%	100%	+15,52 p.p.
Branches	**87,03%**	**96,29%**	**+9,26 p.p.**
Funções	74,19%	100%	+25,81 p.p.
Linhas	**83,50%**	**100%**	**+16,50 p.p.**
`FrequenciaService`: linhas	0%	100%	+100 p.p.
`FrequenciaService`: branches	0%	83,33%	+83,33 p.p.

Os percentuais foram confirmados nos relatórios `coverage-summary.json` arquivados na branch: a v1 cobriu 47/54 branches e 81/97 linhas; a v2 cobriu 52/54 branches e 97/97 linhas. Os logs da nova rodada, em `docs/testes/relatorios/execucao/evolucao-pos-v1/`, confirmam **117/117 testes da v1**, **59/59 da v2** e **build NestJS com código 0**. A execução da suíte geral deve ser validada separadamente; consultar o `RESUMO.md` atualizado, sem usar o resumo antigo de quatro falhas.

Não misturar os escopos: no histórico de quatro arquivos, v1 e v2 empatavam em 100% de linhas e 97,91% de branches. No escopo ampliado de cinco arquivos, a diferença positiva foi gerada pela cobertura adicional de `FrequenciaService` na v2, e não pela perda de testes anteriores na v1.

5. Análise crítica dos resultados

Técnicas e requisitos. Na matriz T01–T48, ambas as suítes já cobriam 21/21 critérios e 48/48 casos. Os EVO-01–EVO-11 expandem a profundidade da US04 sem inventar novos requisitos. A maior quantidade de testes da v1 não é sinônimo de melhores oráculos: ela cobre muitos cenários, mas alguns apenas afirmam que uma exceção qualquer ocorreu.

Linhas versus branches. Uma linha pode ser executada sem percorrer todas as condições booleanas internas. Na medição nova, a v2 chegou a 100% de linhas mas **96,29% de branches**; a cobertura específica de `FrequenciaService` é 83,33% de branches. Uma branch residual do serviço de chamada ocorre perto de um decorator e pode depender da transpilação, hipótese que exige leitura do HTML gerado. Não afirmar cobertura total de decisões do negócio apenas com esses percentuais.

Oráculos e acoplamento. A melhoria da v2 é dupla: oráculos HTTP mais precisos para os 48 casos antigos e novos testes do serviço de frequência. As verificações sobre `where`/`select` do Prisma asseguram filtros importantes, mas podem ser sensíveis a refatorações semanticamente equivalentes. Um teste de integração com banco real daria evidência mais forte da persistência.

Defeitos observados. Nenhum defeito de produção foi confirmado; houve melhoria **de testes**, não correção funcional. Mocks não demonstram transações reais, locks, rollback nem que um aluno estava fisicamente na aula. O código temporário ainda pode ser compartilhado.

6. Conclusão e rastreabilidade

A nova rodada resolve a principal fragilidade demonstrável do protocolo: existe agora uma **melhoria posterior à v1**, com ganhos de **+9,26 pontos percentuais em branches** e **+16,50 em linhas**, sob o mesmo escopo de cinco arquivos. O histórico anterior não foi reescrito: a v2 de 08/11 continua sendo identificada como versão inicial; a v2 ampliada, como rodada de melhoria de 09/11. A extensão confirma a utilidade de gerar cenários com LLM, mas evidencia a necessidade de revisar oráculos, selecionar escopo justo e preservar logs instrumentados.

Apêndices no repositório: `prompt-v1.md`, `llm-log.md`, `matriz-rastreabilidade.md`, `ADENDO-EVOLUCAO-V1-V2.md`, `PLANO-EVOLUCAO-V1-V2.md`, `indicadores-v1-v2.md` e saídas do Vitest. A equipe deve consultar o `RESUMO.md` para verificar a suíte geral; a branch não deve ser integrada se essa etapa apresentar falha. O histórico inicial v2 (08/11) anterior à v1 (09/11) continua sendo uma limitação metodológica explicitada, não uma cronologia corrigida retroativamente. A equipe deverá confirmar a conformidade com o roteiro original antes do merge.